

```

1 -- Task 2: Average progress per video
2
3 -- Select the video id and title, and the average progress ratio for each video, and source
4 SELECT
5     v.CONTENT_ID,
6     -- Classify external_id prefixes to readable source names
7     CASE
8         WHEN c.external_id LIKE 'PGA%' THEN 'AJA'           -- If it starts with PGA → Al Jazeera Arabic
9         WHEN c.external_id LIKE 'PGD%' THEN 'AJD'           -- If it starts with PGD → Al Jazeera Documentary
10        WHEN c.external_id LIKE 'PGV%' THEN 'Atheer'         -- If it starts with PGV → Atheer (Podcasts)
11        WHEN c.external_id LIKE 'PGL%' THEN 'LIVE'           -- If it starts with PGL → Live channels
12        WHEN c.external_id LIKE 'PGC%' THEN 'Originals'       -- If it starts with PGC → Originals / Commissions
13        ELSE 'Other'                                         -- Everything else or missing external_id
14    END AS source, -- name it as source
15
16    -- Use the title from the catalog if available; otherwise use the CONTENT_ID as text
17    --coalesce "take the first non-null value"
18    -- max() because SQL requires an aggregate function when grouping
19    COALESCE(MAX(c.title), CAST(v.CONTENT_ID AS TEXT)) AS title,
20
21    -- Count how many watch events contributed to this average (helps spot small-sample noise)
22    COUNT(*) AS events_count,
23
24    -- Give video's duration (in sec). If multiple values exist, just take the maximum.
25    MAX(v.CONTENT_DURATION) AS content_duration_sec,
26
27    -- Calculate the average progress ratio (Ratio = PROGRESS / CONTENT_DURATION)
28    -- PROGRESS = seconds watched
29    -- CONTENT_DURATION = total video length in seconds
30    -- MIN(100.0, ...) ensures the ratio never goes above 100.0 (100%) (e.g if PROGRESS=900 and
31    -- DURATION=600 → 900*100/600=150.0 → capped to 100.0)
32    AVG(
33        CASE
34            WHEN v.CONTENT_DURATION IS NULL OR v.CONTENT_DURATION = 0 THEN NULL -- Skip rows with
35            -- missing/zero duration
36            ELSE MIN(100.0, (v.PROGRESS * 100.0) / v.CONTENT_DURATION)
37        END
38    ) AS avg_progress_ratio
39 FROM viewership v
40
41 -- Join with the catalog table so we can display the title
42 LEFT JOIN catalog c
43     ON v.CONTENT_ID = c.vod_dve_id
44
45 -- Group results so we get one row per video
46 GROUP BY v.CONTENT_ID
47
48 -- Order results so the videos with the highest average progress appear first
49 ORDER BY avg_progress_ratio DESC;
50

```